

Quiz 1: Video Understanding

04/29/2026

Question 1 (20 points):

After a pooling operation in a CNN, what generally happens to the feature representation?

- A. High-level semantic information increases, while low-level spatial detail decreases.
- B. High-level semantic information decreases, while low-level spatial detail increases.
- C. Both high-level semantic information and low-level spatial detail increase.
- D. Both high-level semantic information and low-level spatial detail decrease.

Question 2 (20 points):

In a CNN, pooling reduces the spatial resolution of feature maps. What is the typical effect of this operation?

- A. It increases spatial precision but reduces semantic abstraction.
- B. It reduces local spatial detail while making the representation more invariant and semantically abstract.
- C. It preserves all pixel-level information while increasing feature map size.
- D. It removes semantic information and keeps only low-level edges.

Question 3: Calculating One Convolution Output Value (60 points)

Given a 5×5 RGB image, its tensor shape is:

$$\text{Input: } H \times W \times C_{in} = 5 \times 5 \times 3$$

where $C_{in} = 3$ corresponds to the three RGB channels: Red, Green, and Blue.

We apply **one 2D convolution filter** with kernel shape:

$$\text{Kernel: } k_H \times k_W \times C_{in} = 3 \times 3 \times 3$$

Equivalently:

$$\text{One filter} = \underbrace{3 \times 3}_{\text{spatial size}} \times \underbrace{3}_{\text{RGB channels}}$$

The convolution settings are:

$$\text{stride} = 1, \text{padding} = 0, b = 1$$

Important note on the kernel shape: This is a 2D convolution on a multi-channel input, not a real 3D convolution.

The kernel slides only along the spatial dimensions H and W . It does **not** slide along the channel axis. The last dimension of the kernel matches the input channel dimension, so the filter has one 3×3 weight matrix for each input channel.

At each spatial location, the filter computes one score from each channel, sums the three channel-wise scores, and then adds the bias.

Please take the following steps to compute the **top-left output activation value**, which corresponds to the input window at **rows 1–3 and columns 1–3** from each channel. In this question, rows and columns are counted starting from 1, unlike Python-style 0-based indexing.

Input image

Red channel	Green channel	Blue channel
$\begin{bmatrix} 1 & 2 & 0 & 1 & 3 \\ 0 & 1 & 3 & 2 & 1 \\ 2 & 1 & 1 & 0 & 2 \\ 1 & 0 & 2 & 3 & 1 \\ 0 & 2 & 1 & 1 & 0 \end{bmatrix}$	$\begin{bmatrix} 0 & 1 & 2 & 0 & 1 \\ 1 & 0 & 1 & 2 & 2 \\ 2 & 2 & 0 & 1 & 0 \\ 1 & 1 & 2 & 0 & 1 \\ 0 & 2 & 1 & 3 & 2 \end{bmatrix}$	$\begin{bmatrix} 1 & 0 & 1 & 2 & 0 \\ 2 & 1 & 0 & 1 & 3 \\ 0 & 1 & 2 & 2 & 1 \\ 1 & 2 & 1 & 0 & 2 \\ 2 & 0 & 1 & 1 & 0 \end{bmatrix}$

Red-channel filter	Green-channel filter	Blue-channel filter
$\begin{bmatrix} 1 & 0 & -1 \\ 1 & 0 & -1 \\ 1 & 0 & -1 \end{bmatrix}$	$\begin{bmatrix} 0 & 1 & 0 \\ 1 & -1 & 1 \\ 0 & 1 & 0 \end{bmatrix}$	$\begin{bmatrix} 1 & 1 & 0 \\ 0 & -1 & 0 \\ 0 & 1 & 1 \end{bmatrix}$

Question

What is the **top-left output activation value**?

Important Note

Answers that provide only the final value without showing the derivation will lose **50 points**, even if the final value is correct.

Your solution must explicitly include:

1. The channel-wise products and sums for Red, Green, and Blue.
2. The summation across channels.
3. The bias addition.

Required calculation steps:

Step 1: Channel-wise element-wise products and sums

For each channel $c \in \{\text{Red}, \text{Green}, \text{Blue}\}$, let X_c^{patch} be the top-left 3×3 patch from that channel.

Compute:

$$S_c = \sum_{i=1}^3 \sum_{j=1}^3 X_c^{\text{patch}}[i, j] \cdot W_c[i, j]$$

where W_c is the 3×3 filter for channel c .

You must show all **9 element-wise products** for each channel, then compute the channel-wise sum.

You should obtain three values:

$$S_R, S_G, S_B$$

Step 2: Sum the channel-wise scores

Add the three channel-wise scores:

$$S = S_R + S_G + S_B$$

Step 3: Add the bias

Finally, add the bias term:

$$y = S + b$$

The result y is the **top-left output activation value**.